

Primeiros Passos com termios.h

Controle de terminal em C — teoria, exemplos práticos e exercícios

Guia de estudo

Contents

Introdução	2
1. O conceito central: modo canônico vs. modo raw	2
1.1 Modo canônico (canonical mode) — o padrão	2
1.2 Modo raw (não-canônico)	2
2. A estrutura struct termios	3
2.1 As duas funções que você sempre usa em par	3
3. As flags mais importantes, uma a uma	4
3.1 c_lflag (local flags) — as que você mais vai usar	4
3.2 c_iflag (input flags)	5
3.3 c_oflag (output flags)	5
3.4 c_cflag (control flags)	5
3.5 c_cc — os caracteres de controle: VMIN e VTIME	5
4. Construindo o modo raw passo a passo	6
5. Exemplo prático: leitura não-bloqueante (VMIN=0, VTIME=0)	8
6. Boas práticas para os projetos do GridForge	9
7. Exercícios	9
Exercício 1 — Leitor de teclas básico	10
Exercício 2 — Sem eco	10
Exercício 3 — Restauração garantida	10
Exercício 4 — Detecção de setas	11
Exercício 5 — Non-blocking com contador	11
Exercício 6 — Simulando VTIME	12
Exercício 7 — Mini editor de um caractere	12
Exercício 8 — Toggle de eco em tempo real	12
Exercício 9 — Módulo reutilizável	13
Exercício 10 — Protótipo de loop de jogo para o GridForge	13
8. Referências para continuar estudando	14

Introdução

Quando você digita em um terminal (o xterm, o terminal do VS Code, o gnome-terminal, etc.), existe uma camada de software entre o seu teclado e o seu programa chamada **driver de terminal (tty driver)**. Por padrão, esse driver faz um trabalho “invisível” enorme por você: junta os caracteres digitados em uma linha, permite apagar com Backspace, só entrega a linha ao programa quando você aperta Enter, e ecoa (mostra) o que você digitou na tela.

Isso é ótimo para um programa comum, mas é péssimo se você quer construir algo como:

- Um editor de pixel art onde as setas do teclado devem mover o cursor **imediatamente**, sem esperar o Enter.
- O Jogo da Vida (Game of Life) rodando em tempo real, onde você quer capturar uma tecla para pausar, sem travar o programa esperando uma linha inteira.
- Qualquer visualizador em terminal que precise ler tecla-a-tecla, sem eco, sem espera.

O arquivo de cabeçalho `<termios.h>` é a porta de entrada em C (POSIX) para conversar diretamente com esse driver de terminal e mudar esse comportamento. É exatamente o que você precisa para os projetos do GridForge.

Este guia assume **zero conhecimento prévio de POSIX**. Vamos construir o entendimento em camadas, sempre com código que compila e roda.

1. O conceito central: modo canônico vs. modo raw

Existem, na prática, dois “modos” de terminal que você precisa entender:

1.1 Modo canônico (canonical mode) — o padrão

É o modo em que o terminal já vem configurado quando seu programa começa a rodar. Características:

- O terminal **armazena a linha inteira** enquanto você digita (isso se chama *line buffering*).
- Seu programa só recebe os dados quando você aperta **Enter**.
- Backspace, Ctrl+U (apagar linha) e outras teclas de edição funcionam automaticamente, tratadas pelo próprio driver.
- Tudo que você digita é **ecoado** (mostrado) na tela automaticamente.

Isso é o que faz `scanf("%d", &x)` funcionar naturalmente: você digita 42, edita se errar, aperta Enter, e só então o programa recebe.

1.2 Modo raw (não-canônico)

- Cada tecla chega ao seu programa **imediatamente**, um caractere por vez, sem esperar Enter.

- Não há edição automática (Backspace vira só um caractere qualquer, 0x7f, que seu programa precisaria tratar manualmente se quisesse).
- O eco pode ser desligado, para você controlar exatamente o que aparece na tela.
- Sinais como Ctrl+C (que normalmente mata o programa) podem ser desativados, permitindo que seu próprio código decida o que fazer com essas teclas.

`termios.h` é a ferramenta que permite ligar e desligar, bit a bit, essas características.

2. A estrutura `struct termios`

Toda a configuração do terminal vive dentro de uma única struct:

```
#include <termios.h>

struct termios {
    tcflag_t c_iflag;    // flags de entrada (input)
    tcflag_t c_oflag;    // flags de saída (output)
    tcflag_t c_cflag;    // flags de controle (control)
    tcflag_t c_lflag;    // flags "locais" (local) – a mais importante no dia a dia
    cc_t      c_cc[NCCS]; // caracteres de controle especiais (ex: VMIN, VTIME)
    // pode haver outros campos dependendo do sistema (ex: velocidade da linha),
    // mas os 5 acima são os que você vai manipular.
};
```

Pense nela como um painel de interruptores. Cada `c_*flag` é um inteiro onde cada **bit** liga ou desliga um comportamento específico. Você nunca escreve o número inteiro na mão — usa constantes com nomes (macros), como `ECHO`, `ICANON`, `ISIG`, combinadas com operadores bit a bit (`&`, `|`, `~`).

2.1 As duas funções que você sempre usa em par

```
int tcgetattr(int fd, struct termios *termios_p);
int tcsetattr(int fd, int optional_actions, const struct termios *termios_p);
```

- **`tcgetattr`**: lê a configuração *atual* do terminal para dentro da sua struct. Você sempre começa lendo o estado atual — nunca inventa uma struct do zero.
- **`tcsetattr`**: aplica a struct (já modificada por você) de volta ao terminal.

`fd` é o descritor de arquivo do terminal — quase sempre `STDIN_FILENO` (0), porque você está mudando como a entrada padrão se comporta.

O parâmetro `optional_actions` de `tcsetattr` diz **quando** aplicar a mudança:

Constante	Significado
<code>TCSANOW</code>	Aplica imediatamente.
<code>TCSADRAIN</code>	Espera todo o output pendente ser escrito antes de aplicar.

Constante	Significado
TCSAFLUSH	Espera o output ser escrito e descarta qualquer input ainda não lido. É a opção mais usada ao entrar em modo raw, para não “sobrar” teclas digitadas antes da mudança.

Exemplo mínimo: ler e devolver sem mudar nada

```
#include <stdio.h>
#include <termios.h>
#include <unistd.h>

int main(void) {
    struct termios original;

    if (tcgetattr(STDIN_FILENO, &original) == -1) {
        perror("tcgetattr");
        return 1;
    }

    printf("Configuracao atual lida com sucesso.\n");
    printf("c_lflag = %u\n", (unsigned) original.c_lflag);

    return 0;
}
```

Esse programa não muda nada ainda — só prova que você consegue ler a configuração. Compile com:

```
gcc -o teste teste.c
./teste
```

3. As flags mais importantes, uma a uma

3.1 c_lflag (local flags) — as que você mais vai usar

Flag	O que controla	Ligada (padrão)	Desligada
ECHO	Ecoa (mostra) o que é digitado	Você vê o que digita	Nada aparece — útil para senha ou desenho customizado
ICANON	Modo canônico (buffer de linha)	Espera Enter	Cada tecla chega na hora

Flag	O que controla	Ligada (padrão)	Desligada
ISIG	Gera sinais para Ctrl+C (SIGINT), Ctrl+Z (SIGTSTP)	Ctrl+C mata o programa	Ctrl+C vira só um byte comum (0x03) que seu código lê
IEXTEN	Habilita processamento estendido (ex: Ctrl+V)	Ligada	Desligada evita interferências em modo raw

3.2 c_iflag (input flags)

Flag	O que controla
IXON	Controle de fluxo por software (Ctrl+S / Ctrl+Q pausam/retomam saída)
ICRNL	Converte \r (Enter físico, carriage return) recebido em \n (newline)
BRKINT	Um “break” no sinal gera SIGINT
INPCK, ISTRIP	Checagem e strip de paridade (bits legados, raramente necessários hoje)

3.3 c_oflag (output flags)

Flag	O que controla
OPOST	Pós-processamento de saída (ex: transformar \n em \r\n ao imprimir)

3.4 c_cflag (control flags)

Controla propriedades de “hardware” da linha serial (tamanho de caractere, paridade). No dia a dia de um terminal comum em Linux, você raramente mexe aqui, exceto por:

Flag	O que controla
CS8	Usa 8 bits por caractere (o padrão desejado quase sempre)

3.5 c_cc — os caracteres de controle: VMIN e VTIME

Esses dois só têm efeito quando ICANON está **desligado** (modo raw), e controlam como read() se comporta:

c_cc[VMIN]	c_cc[VTIME]	Comportamento de read()
0	0	Não-bloqueante: retorna na hora, mesmo sem nenhum byte disponível
0	> 0	Espera até VTIME décimos de segundo por <i>qualquer</i> byte; se nada chegar, retorna 0
≥ 1	0	Bloqueia até ter pelo menos VMIN bytes disponíveis
≥ 1	> 0	Bloqueia até ter VMIN bytes, ou até VTIME décimos de segundo se passarem entre bytes (timer entre-bytes)

Essa tabela é a peça-chave para quem quer, por exemplo, ler uma tecla, mas continuar atualizando a tela de um jogo se nenhuma tecla for pressionada (VMIN = 0, VTIME = 1, por exemplo).

4. Construindo o modo raw passo a passo

A receita clássica para “modo raw” combina o que vimos:

```
#include <stdio.h>
#include <stdlib.h>
#include <termios.h>
#include <unistd.h>

static struct termios g_original_termios;

void restaurar_terminal(void) {
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &g_original_termios);
}

void ativar_modos_raw(void) {
    struct termios raw;

    // 1. Guarda a configuracao original para poder restaurar depois
    if (tcgetattr(STDIN_FILENO, &g_original_termios) == -1) {
        perror("tcgetattr");
        exit(1);
    }
}
```

```

// 2. Garante que a configuracao original volte ao sair do programa,
// mesmo que seja por exit() em outro lugar do codigo
atexit(restaurar_terminal);

raw = g_original_termios;

// 3. Desliga eco, modo canonico, sinais (Ctrl+C) e processamento estendido
raw.c_lflag &= ~(ECHO | ICANON | ISIG | IEXTEN);

// 4. Desliga conversao de CR em NL e controle de fluxo Ctrl+S/Ctrl+Q
raw.c_iflag &= ~(ICRNL | IXON);

// 5. Desliga pos-processamento de saida
raw.c_oflag &= ~(OPOST);

// 6. read() retorna assim que 1 byte estiver disponivel, sem timeout
raw.c_cc[VMIN] = 1;
raw.c_cc[VTIME] = 0;

// 7. Aplica a nova configuracao
if (tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw) == -1) {
    perror("tcsetattr");
    exit(1);
}
}

int main(void) {
    ativar_modos_raw();

    printf("Modo raw ativo. Digite teclas ('q' para sair):\r\n");

    char c;
    while (read(STDIN_FILENO, &c, 1) == 1) {
        if (c == 'q') break;

        if (c >= 32 && c < 127) {
            printf("Tecla: '%c' (codigo %d)\r\n", c, c);
        } else {
            printf("Tecla de controle (codigo %d)\r\n", c);
        }
    }

    printf("Encerrando, terminal restaurado.\r\n");
    return 0;
    // atexit() cuida de chamar restaurar_terminal() aqui
}

```

Alguns pontos que costumam confundir iniciantes:

1. **Por que `\r\n` e não só `\n`?** Como 0POST está desligado, o terminal não converte mais `\n` em `\r\n` sozinho. Sem o `\r`, o cursor pula de linha mas não volta ao início — o texto “escadinha”. Em modo raw, você é responsável por isso.
 2. **Por que `atexit()`?** Se o seu programa travar de forma inesperada, ou usar `exit()` no meio do código, você quer garantir que o terminal do usuário **não fique quebrado** (sem eco, sem Enter funcionando) depois que o programa fechar. Isso é a causa nº 1 de “meu terminal ficou estranho depois de rodar o programa” — sempre restaure o estado original.
 3. **Por que salvar `g_original_termios` antes de mudar qualquer coisa?** Porque `tcsetattr` sobrescreve a configuração. Sem guardar o original, não haveria como voltar ao estado anterior.
-

5. Exemplo prático: leitura não-bloqueante (VMIN=0, VTIME=0)

Útil para um loop de jogo/visualizador que precisa continuar rodando (atualizando a tela, avançando uma simulação) mesmo se nenhuma tecla for pressionada:

```
#include <stdio.h>
#include <termios.h>
#include <unistd.h>

void ativar_nao_bloqueante(struct termios *original) {
    struct termios raw;
    tcgetattr(STDIN_FILENO, original);
    raw = *original;

    raw.c_lflag &= ~(ECHO | ICANON);
    raw.c_cc[VMIN] = 0; // nao espera nenhum byte
    raw.c_cc[VTIME] = 0; // nao espera nenhum tempo

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

int main(void) {
    struct termios original;
    ativar_nao_bloqueante(&original);

    int passo = 0;
    char c;
    while (1) {
        ssize_t n = read(STDIN_FILENO, &c, 1);

        if (n == 1 && c == 'q') break;
    }
}
```



```

    if (n == 1) {
        printf("\rTecla capturada: %c    (passo %d)    ", c, passo);
    } else {
        printf("\rSimulando passo %d... (nenhuma tecla)    ", passo);
    }
    fflush(stdout);

    passo++;
    usleep(200000); // 200ms - simula um "tick" de simulacao/jogo
}

tcsetattr(STDIN_FILENO, TCSAFLUSH, &original);
printf("\nFim.\n");
return 0;
}

```

Repare que aqui **não** desligamos ISIG, então Ctrl+C ainda mata o programa normalmente — o que é uma escolha razoável para um primeiro protótipo (evita precisar restaurar o terminal manualmente enquanto você testa).

6. Boas práticas para os projetos do GridForge

- **Sempre** guarde a struct `termios` original antes de qualquer modificação, e restaure-a (via `atexit`, ou tratando `SIGINT/SIGTERM`) antes de sair — inclusive em caminhos de erro.
- Prefira `TCSAFLUSH` ao ativar modo raw, para descartar teclas já digitadas antes da troca de modo.
- Separe a lógica de terminal em seu próprio módulo (ex.: `src/terminal.c` + `include/terminal.h`), com funções como `terminal_ativar_raw()` e `terminal_restaurar()` — isso conversa bem com a arquitetura modular que você já usa nos outros projetos.
- Ao desligar `OP0ST`, lembre-se de usar `\r\n` (ou uma função de “limpar linha” própria) em vez de só `\n`.
- Teste sempre o valor de retorno de `tcgetattr/tcsetattr` (-1 indica erro) e use `perror` para depurar.

7. Exercícios

Tente resolver cada um compilando e testando de verdade no seu terminal Linux antes de olhar qualquer solução pronta. Cada exercício traz: o que fazer, um passo a passo, e o objetivo de aprendizado por trás dele.

Exercício 1 — Leitor de teclas básico

O que fazer: escreva um programa que, assim que você aperta uma tecla, mostra o código ASCII dela na tela — sem precisar apertar Enter. Pare quando a tecla q for pressionada.

Passo a passo:

1. Salve a configuração original do terminal com `tcgetattr` (você vai precisar dela para restaurar depois).
2. Copie essa configuração para uma nova struct e desligue `ICANON` (para não esperar Enter).
3. Aplique a nova configuração com `tcsetattr`.
4. Em loop, leia um caractere por vez com `read(STDIN_FILENO, &c, 1)`.
5. Para cada caractere lido, imprima o valor **numérico** dele, não a letra — use `printf("Codigo: %d\n", c)`, não `%c`.
6. Se o caractere lido for 'q', saia do loop (`break`).
7. Ao sair do loop, restaure a configuração original com `tcsetattr`, usando a struct salva no passo 1.

Objetivo: praticar o ciclo completo `tcgetattr` → modificar → `tcsetattr` → ler → restaurar, sem se preocupar ainda com aparência na tela. Esse ciclo é a base de todos os exercícios seguintes.

Ponto de atenção: 'a' tem código 97, 'A' tem 65, Enter costuma ser 13 ou 10, espaço é 32. Se você imprimir com `%c` em vez de `%d`, vai ver a letra, não o código — releia o enunciado se isso acontecer.

Exercício 2 — Sem eco

O que fazer: pegue o programa do exercício 1 e desligue também `ECHO`, de forma que a tecla digitada não apareça sozinha na tela.

Passo a passo:

1. Reaproveite o código do exercício 1.
2. No passo de modificar a struct, desligue `ECHO` também: `raw.c_lflag &= ~(ICANON | ECHO);`
3. Rode o programa e digite algo — nada deve aparecer na tela automaticamente.
4. Adicione um `printf` próprio, tipo `"Recebi a tecla, codigo: %d\n"`, para confirmar que o programa está lendo mesmo sem eco.

Objetivo: entender que eco (o que a *tela* mostra) e leitura (o que o *programa* recebe) são duas coisas completamente separadas — uma não depende da outra.

Exercício 3 — Restauração garantida

O que fazer: garanta que o terminal volta ao estado normal em três cenários diferentes.

Passo a passo:

1. Cenário A: o programa termina o `main()` normalmente (usando `return 0;`) — confirme que o terminal está normal depois.
2. Cenário B: force uma saída no meio do código com `exit(1)`, simulando um erro — confirme que o terminal *ainda assim* volta ao normal (é aqui que `atexit()` mostra seu valor).
3. Cenário C (opcional, mais avançado): use `signal(SIGINT, seu_handler)` para capturar o Ctrl+C, e dentro do handler chame a função de restaurar o terminal antes de encerrar com `exit()`.
4. Em todos os cenários, teste digitando `stty -a` no terminal depois de rodar o programa, para conferir que a configuração é igual à de antes.

Objetivo: garantir que nenhum caminho de saída do programa deixe o terminal do usuário quebrado (sem eco, sem Enter funcionando). Esse é o erro mais comum em programas que mexem em `termios` — e o motivo de sempre usarmos `atexit()` nos exemplos deste guia.

Exercício 4 — Detecção de setas

O que fazer: detecte quando o usuário aperta uma seta do teclado (cima, baixo, esquerda, direita) e imprima qual foi.

Passo a passo:

1. Ative modo raw (como no exercício 1).
2. Em loop, leia 1 byte. Se o byte lido for 27 (ESC), isso é o início de uma sequência de escape — não é uma tecla ESC “pura” ainda.
3. Quando detectar 27, leia mais 2 bytes: o primeiro deve ser 91 (o caractere `[]`); o segundo será 65 (A = cima), 66 (B = baixo), 67 (C = direita) ou 68 (D = esquerda).
4. Baseado nesse terceiro byte, imprima “Seta para cima”, “Seta para baixo”, etc.
5. Se o primeiro byte não for 27, trate como uma tecla normal (igual ao exercício 1).

Objetivo: primeiro contato com sequências de escape ANSI, que aparecem toda vez que uma tecla especial (seta, Home, End, F1-F12) é pressionada. Isso é essencial para qualquer visualizador de terminal navegável.

Exercício 5 — Non-blocking com contador

O que fazer: um contador que incrementa sozinho a cada ~100ms, mas que reage instantaneamente se o espaço for apertado.

Passo a passo:

1. Configure o modo raw com `VMIN=0`, `VTIME=0` (leitura que nunca bloqueia, como no exemplo da Seção 5 do guia).
2. Em loop: tente ler 1 byte com `read()`. Se o retorno for 1 (leu algo) e o caractere for espaço (`' '`), some 10 ao contador.
3. Independente de ter lido algo ou não, incremente o contador em 1 e imprima seu valor (use `\r` no início do `printf` para reescrever a mesma linha, e `fflush(stdout)` depois).

4. Chame `usleep(100000)` (100ms) a cada volta do loop, para não consumir CPU à toa.
5. Pare quando `q` for lido.

Objetivo: simular o “tick” de um jogo ou simulação, que continua avançando independente do teclado — a base de qualquer loop de animação em terminal, como os do GridForge.

Exercício 6 — Simulando VTIME

O que fazer: um programa que espera até 1 segundo por uma tecla; se nada chegar nesse tempo, avisa e tenta de novo.

Passo a passo:

1. Configure `VMIN=0`, `VTIME=10` (10 décimos de segundo = 1 segundo).
2. Em loop, chame `read(STDIN_FILENO, &c, 1)` e guarde o valor retornado (`ssize_t n = read(...)`).
3. Se `n == 1`, uma tecla chegou dentro do 1 segundo — imprima qual foi.
4. Se `n == 0`, nenhuma tecla chegou a tempo — imprima “tempo esgotado” e volte ao início do loop (o `read` vai esperar outro 1 segundo).
5. Pare o loop quando a tecla lida for `q`.

Objetivo: comparar na prática a diferença entre não esperar nada (exercício 5, `VTIME=0`) e esperar um tempo limitado antes de desistir (`VTIME=10`).

Exercício 7 — Mini editor de um caractere

O que fazer: simule o comportamento de apagar com Backspace, que o modo canônico faz sozinho mas que em modo raw fica por sua conta.

Passo a passo:

1. Ative modo raw, com eco desligado (`ECHO` fora).
2. Em loop, leia um caractere.
3. Se o código lido for 127 (Backspace), imprima a sequência “\b \b” — isso move o cursor um espaço para trás, escreve um espaço em branco por cima (apagando visualmente o que estava lá) e move o cursor para trás de novo.
4. Se for qualquer outro caractere imprimível (entre 32 e 126), imprima o próprio caractere na tela (já que o eco está desligado, você é quem decide mostrá-lo).
5. Pare quando Enter (código 13 ou 10) for pressionado.

Objetivo: perceber que, em modo raw, comportamentos que pareciam “de graça” (como apagar com Backspace) precisam ser reimplementados manualmente — o driver de terminal não faz mais isso por você.

Exercício 8 — Toggle de eco em tempo real

O que fazer: ligar e desligar o eco do terminal *durante* a execução do programa, sem sair do modo raw.

Passo a passo:

1. Ative o modo raw começando com ECHO desligado.
2. Em loop, leia um caractere.
3. Se o caractere lido for 'e', chame `tcgetattr` de novo, inverta o bit ECHO na struct (ligue se estava desligado, desligue se estava ligado) e aplique com `tcsetattr` novamente — tudo isso *dentro* do loop, sem sair do programa.
4. Imprima uma mensagem confirmando o novo estado (“Eco ligado” / “Eco desligado”).
5. Pare quando q for pressionado (tomando cuidado para não confundir com o 'e' do toggle).

Objetivo: entender que a configuração do terminal não é fixa — pode ser trocada quantas vezes forem necessárias durante a execução, não só uma vez no início do programa.

Exercício 9 — Módulo reutilizável

O que fazer: reorganizar o código de algum exercício anterior em um módulo de terminal próprio, seguindo a arquitetura que você já usa nos outros projetos.

Passo a passo:

1. Crie `include/terminal.h` com os protótipos das funções, por exemplo: `void terminal_ativar_raw(void);, void terminal_restaurar(void);, int terminal_ler_tecla(void);.`
2. Crie `src/terminal.c` com a implementação de cada função (a struct original pode ficar como uma variável estática dentro desse arquivo, como no exemplo da Seção 4).
3. No `main.c`, inclua apenas `"terminal.h"` e chame as funções — nenhuma chamada direta a `tcgetattr/tcsetattr` deve aparecer em `main.c`.
4. Compile os dois arquivos juntos: `gcc -Iinclude src/main.c src/terminal.c -o programa.`
5. Teste que o comportamento é idêntico ao do exercício original.

Objetivo: aplicar a arquitetura modular (`src/` + `include/`, zero lógica fora de funções dedicadas em `main.c`) a este código novo, do mesmo jeito que você já organiza os outros projetos do GridForge.

Exercício 10 — Protótipo de loop de jogo para o GridForge

O que fazer: juntar tudo em um único programa que sirva de ponto de partida para qualquer visualizador do GridForge.

Passo a passo:

1. Use o módulo de terminal do exercício 9 como base.
2. Configure o modo raw com `VMIN=0`, `VTIME=0` (não-bloqueante, exercício 5).
3. Adicione a detecção de setas do exercício 4 dentro da função de leitura do módulo.
4. No loop principal (`main.c`), avance algo simples a cada iteração — por exemplo, um contador de “passos de simulação” — independente de haver tecla pressionada ou não.

5. Trate a tecla `q` para encerrar o loop.
6. Garanta que `terminal_restaurar()` é chamada ao final, incluindo por `atexit()` (exercício 3).

Objetivo: ter, ao final, um esqueleto de entrada funcional — modo raw, leitura não-bloqueante, setas, tecla de saída e restauração garantida — pronto para ser reaproveitado no primeiro projeto real do GridForge (Game of Life, Pathfinding Visualizer, Sandpile, etc.).

8. Referências para continuar estudando

- `man 3 termios` — a referência oficial completa, agora que você já tem o vocabulário para entender melhor cada campo.
- `man 3 tcgetattr` e `man 3 tcsetattr`.
- Capítulo de terminal I/O do livro *Advanced Programming in the UNIX Environment* (Stevens & Rago), se quiser se aprofundar em sinais, grupos de processo e terminais controladores.